

```
from telegram import Update from telegram import KeyboardButton, ReplyKeyboardMarkup

from telegram.ext import ( ApplicationBuilder, ContextTypes, ConversationHandler, MessageHandler,
CommandHandler, filters )

TOKEN = "YOUR_BOT_TOKEN"
```

مراحل ثبت نام

```
NAME, FAMILY, AGE, CITY = range(4)
```

کیبورد اصلی

```
keyboard = [ [KeyboardButton("ثبت نام")], [KeyboardButton("راهنما")], [KeyboardButton("درباره ما")] ]

markup = ReplyKeyboardMarkup( keyboard, resize_keyboard=True )
```

منوی اصلی

```
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE): await update.message.reply_text( "به 🖐️ ربات خوش آمدی", reply_markup=markup )

async def menu_handler(update: Update, context: ContextTypes.DEFAULT_TYPE): text =
update.message.text
```

```
if text == "راهنما":
    await update.message.reply_text(
        ""
```

دستورات:

```
/start /cancel "" )
```

```
elif text == "درباره ما":
    await update.message.reply_text(
```

```
    ". این ربات برای تمرین تلگرام بات ساخته شده است"  
)
```

شروع ثبت نام

```
async def start_conv(update: Update, context: ContextTypes.DEFAULT_TYPE): await  
update.message.reply_text("اسمت چیه؟") return NAME
```

دریافت نام

```
async def get_name(update: Update, context: ContextTypes.DEFAULT_TYPE): name = update.message.text
```

```
    if name.isalpha():  
        context.user_data["name"] = name  
        await update.message.reply_text("فامیلت؟")  
        return FAMILY  
  
    await update.message.reply_text(  
        ". برای وارد کردن اسم فقط از حروف الفبا استفاده کنید"  
    )  
    return NAME
```

دریافت فامیلی

```
async def get_family(update: Update, context: ContextTypes.DEFAULT_TYPE): family = update.message.text
```

```

if family.isalpha():
    context.user_data["family"] = family
    await update.message.reply_text("چند سالته؟")
    return AGE

await update.message.reply_text(
    "برای وارد کردن فامیلی فقط از حروف الفبا استفاده کنید"
)
return FAMILY

```

دریافت سن

async def get_age(update: Update, context: ContextTypes.DEFAULT_TYPE): age = update.message.text

```

if age.isdigit() and 0 < int(age) < 120:
    context.user_data["age"] = age
    await update.message.reply_text("اسم شهرت چیه؟")
    return CITY

await update.message.reply_text(
    "سن باید فقط عدد و در بازه مناسب باشد"
)
return AGE

```

دریافت شهر

async def get_city(update: Update, context: ContextTypes.DEFAULT_TYPE): city = update.message.text

```

if city.isalpha():
    context.user_data["city"] = city

```

```
await update.message.reply_text(
    f"نام"
```

✔️ ثبت نام با موفقیت انجام شد

👤 نام: {context.user_data["name"]} 👤 فامیلی: {context.user_data["family"]} 🎂 سن: {context.user_data["age"]}
🏠 شهر: {context.user_data["city"]} "")

```
return ConversationHandler.END
```

```
await update.message.reply_text(
    "اسم شهر فقط باید از حروف الفبا تشکیل شده باشد"
)
return CITY
```

لغو ثبت نام

```
async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE): await
update.message.reply_text("ثبت نام لغو شد ❌") return ConversationHandler.END
```

ساخت برنامه

```
app = ApplicationBuilder().token(TOKEN).build()
```

```
conv_handler = ConversationHandler( entry_points=[ MessageHandler( filters.Regex("^ثبت نام$"),
start_conv ) ],
```

```
states={
    NAME: [
        MessageHandler(
```

```

        filters.TEXT & ~filters.COMMAND,
        get_name
    )
],

FAMILY: [
    MessageHandler(
        filters.TEXT & ~filters.COMMAND,
        get_family
    )
],

AGE: [
    MessageHandler(
        filters.TEXT & ~filters.COMMAND,
        get_age
    )
],

CITY: [
    MessageHandler(
        filters.TEXT & ~filters.COMMAND,
        get_city
    )
]
},

fallbacks=[
    CommandHandler("cancel", cancel)
]

```

)

app.add_handler(CommandHandler("start", start))

app.add_handler(conv_handler)

app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, menu_handler))

print("ریات روشن شد")

app.run_polling()